**Universidad de San Carlos de Guatemala**
**Centro Universitario de Occidente**
**División de Ingeniería**
**Carrera: Ingeniería en Ciencias y Sistemas**
**Curso: Laboratorio, IPC2**
**Auxiliar: Roberto Bautista**
**Docente: Ing. Daniel González**

# Trabajo: Manual Técnico, practica 1

**Estudiante: Jefferson Salvador Gómez Menchú**
**jeffersongomez201830892@cunoc.edu.gt**

**Registro académico: 201830892**

**Quetzaltenango 11-03-2025**

# Objetivos

## Objetivos Generales

- Aplicar conceptos de bases de datos relacionales en el desarrollo de un videojuego.
- Aplicar programación orientada a objetos en la modelación de entidades del juego.
- Implementar interfaces gráficas interactivas utilizando Java Swing.
- Desarrollar la lógica necesaria para gestionar estados, puntajes y flujo de juego.

## Objetivos específicos

- Modelar la información del videojuego mediante diagramas Entidad/Relación.
- Diseñar e implementar un esquema de base de datos relacional.
- Desarrollar una aplicación conectada a base de datos.
- Implementar un sistema de puntuación basado en el desempeño del jugador.
- Validar la información ingresada y los estados del juego.
- Aplicar buenas prácticas de programación orientada a objetos.

**Explicación de la Arquitectura y Tecnologías utilizadas**

**Arquitectura:**

1. Capa de presentación: Interfaz gráfica que permite al usuario interactuar con el sistema.
2. Capa de control: Coordina las operaciones entre la interfaz y la base de datos.
3. Capa de peticiónes/Querys: Implementa las Querys a la base de datos para poder ingresar, actualizar o eliminar información que se requiera.
4. Capa de modelo: Define las estructuras (entidades) de datos utilizadas (Rol, Usuario, Producto, etc.).

La arquitectura está diseñada para facilitar la extensibilidad y mantenibilidad del código, permitiendo modificar una capa sin afectar significativamente a las demás.

**Tencologías utilizadas:**

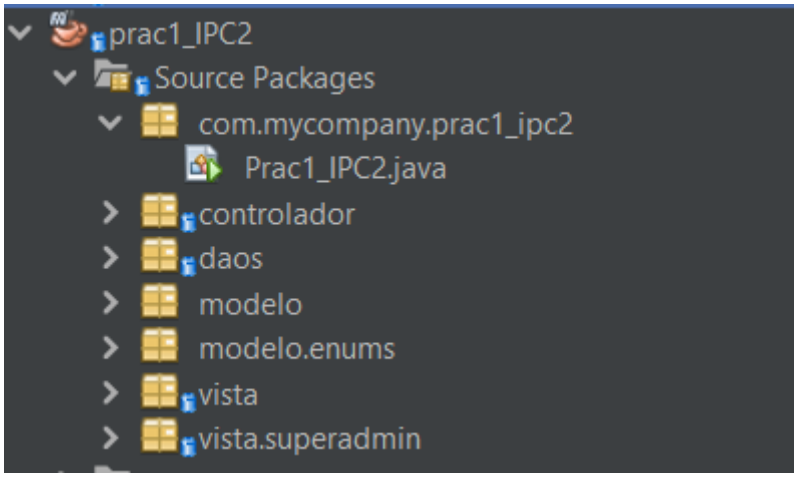
- **Java: Lenguaje de programación principal (versión 19 o superior)**
- **MySQL: Generador de base de datos con MariaDB en su versión 11.7**
- **Swing: Biblioteca gráfica para la interfaz de usuario**

**Requisitos del entorno de desarrollo:**

- **JDK 8 o superior**
- **IDE compatible con Java (Eclipse, IntelliJ IDEA, NetBeans) ó Visual Studio Code**
- **MariaDB o cualquier otro gestor de base de datos instalado para generación del Script y el mapeo de la base de datos.**

## Estructura del proyecto

El proyecto está organizado en los siguientes paquetes:




**Descripción de los principales componentes**:

- **Main.java**: Prac1_IPC2.java., punto de entrada de la aplicación
- **controlador**: Clases que gestionan la lógica de la aplicación
- **modelo**: Define las estructuras de datos utilizadas
    - **enums**: Acá se representan estados importantes que se usan recurrentemente en la aplicación.
- **daos**: Genera comunicación con la base de datos
- **vista**: Implementa la interfaz gráfica de usuario
    - **superadmin**: acá se implementa la interfaz especifica del Rol de super administrador.

## Archivo POM.xl para la correcta comunicación de Java conMySQL

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLS
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.mycompany</groupId>
    <artifactId>prac1_IPC2</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>jar</packaging>
    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <maven.compiler.source>21</maven.compiler.source>
        <maven.compiler.target>21</maven.compiler.target>
        <exec.mainClass>com.mycompany.prac1_ipc2.Prac1_IPC2</exec.mainClass>
    </properties>
    <dependencies>
        <dependency>
            <groupId>mysql</groupId>
            <artifactId>mysql-connector-java</artifactId>
            <version>8.0.33</version>
        </dependency>
    </dependencies>
</project>
```

## Base de datos

| # | Tabla | Descripcion |
|---|---|---|
| 1 | roles | Tipos de usuario: JUGADOR, ADMIN_TIENDA, SUPER_ADMIN |
| 2 | sucursales | Locales fisicos |
| 3 | usuarios | Todos los usuarios del sistema |
| 4 | productos | Catalogo global de productos del menú |
| 5 | producto_sucursal | Disponibilidad de productos por sucursal |
| 6 | parametros_juego | configuración global del juego en clave-valor |
| 7 | partidas | Sesiones de juego de cada jugador |
| 8 | niveles_partida | Registro de cada nivel alcanzado por partida |
| 9 | pedidos | Pedidos generados durante una partida |
| 10 | detalle_pedido | Productos incluidos en cada pedido |
| 11 | historial_estados_pedido | cambios de estado por pedido |

A continuacion se brinda el script utilizado para poder mapear la base de datos, por medio de estas tablas se logra la relación y su posterior transformación a codigo y a logica del funcionamiento.

```sql
CREATE DATABASE practica1_IPC2;
USE practica1_IPC2;

CREATE TABLE roles (
    id_rol          INT AUTO_INCREMENT PRIMARY KEY,
    nombre          VARCHAR(100)  NOT NULL UNIQUE,
    descripcion     VARCHAR(255),
    activo          BOOLEAN       NOT NULL DEFAULT TRUE
);
ALTER TABLE roles DROP COLUMN fecha_creacion;
ALTER TABLE roles DROP COLUMN fecha_modificacion;

CREATE TABLE sucursales (
    id_sucursal     INT AUTO_INCREMENT PRIMARY KEY,
    nombre          VARCHAR(100) NOT NULL,
    direccion       VARCHAR(250),
    activa          BOOLEAN       NOT NULL DEFAULT TRUE
);
ALTER TABLE sucursales DROP COLUMN telefono;

CREATE TABLE usuarios (
    id_usuario      INT AUTO_INCREMENT PRIMARY KEY,
    usuario       VARCHAR(50)  NOT NULL UNIQUE,
    contraseña   VARCHAR(255) NOT NULL,
    nombre          VARCHAR(100) NOT NULL,
    apellido        VARCHAR(100),
    email           VARCHAR(100) UNIQUE,
    id_rol          INT NOT NULL,
    id_sucursal     INT,
    activo          BOOLEAN       NOT NULL DEFAULT TRUE,
    CONSTRAINT fk_usuario_rol      FOREIGN KEY (id_rol)      REFERENCES roles(id_rol),
```

```sql
CREATE TABLE productos (
    id_producto     INT AUTO_INCREMENT PRIMARY KEY,
    nombre          VARCHAR(100) NOT NULL,
    descripcion     VARCHAR(250),
    precio          DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    categoria       VARCHAR(80),
    imagen          VARCHAR(255),
    activo          BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE producto_sucursal (
    id_producto_sucursal INT AUTO_INCREMENT PRIMARY KEY,
    id_producto     INT NOT NULL,
    id_sucursal     INT NOT NULL,
    disponible      BOOLEAN NOT NULL DEFAULT TRUE,
    FOREIGN KEY (id_producto) REFERENCES productos(id_producto),
    FOREIGN KEY (id_sucursal) REFERENCES sucursales(id_sucursal)
);

CREATE TABLE parametros_juego (
    id_parametro    INT AUTO_INCREMENT PRIMARY KEY,
    clave           VARCHAR(100)  NOT NULL UNIQUE,
    valor           VARCHAR(250) NOT NULL,
    descripcion     VARCHAR(255)
);
```

```sql
CREATE TABLE partida (
    id_partida            INT AUTO_INCREMENT PRIMARY KEY,
    id_usuario            INT NOT NULL,
    id_sucursal           INT NOT NULL,
    fecha_inicio          DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    fecha_fin             DATETIME,
    estado                ENUM('EN_CURSO','FINALIZADA','ABANDONADA') NOT NULL DEFAULT 'EN_CURSO',
    puntaje_total         INT NOT NULL DEFAULT 0,
    nivel_maximo          INT NOT NULL DEFAULT 1,
    pedidos_completados   INT NOT NULL DEFAULT 0,
    pedidos_cancelados    INT NOT NULL DEFAULT 0,
    pedidos_no_entregados INT NOT NULL DEFAULT 0,
    FOREIGN KEY (id_usuario)  REFERENCES usuarios(id_usuario),
    FOREIGN KEY (id_sucursal) REFERENCES sucursales(id_sucursal)
);
```

```sql
CREATE TABLE niveles_partida (
    id_nivel_partida     INT AUTO_INCREMENT PRIMARY KEY,
    id_partida           INT NOT NULL,
    nivel                INT NOT NULL,
    fecha_alcanzado      DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    puntaje_por_alcanzar  INT NOT NULL DEFAULT 0,
    pedidos_por_alcanzar  INT NOT NULL DEFAULT 0,
    FOREIGN KEY (id_partida) REFERENCES partida(id_partida)
);
```

```sql
CREATE TABLE pedidos (
    id_pedido          INT AUTO_INCREMENT PRIMARY KEY,
    id_partida         INT NOT NULL,
    numero_pedido      INT NOT NULL,
    nivel_al_crear     INT NOT NULL DEFAULT 1,
    tiempo_limite_seg  INT NOT NULL,
    fecha_creacion     DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    fecha_finalizacion DATETIME,
    estado             ENUM('RECIBIDA','PREPARANDO','EN_HORNO','LISTA',
                           'CANCELADA','NO_ENTREGADO') NOT NULL DEFAULT 'RECIBIDA',
    tiempo_usado_seg   INT,
    puntos_obtenidos   INT NOT NULL DEFAULT 0,
    FOREIGN KEY (id_partida) REFERENCES partida(id_partida)
);
```

```sql
CREATE TABLE detalle_pedido (
    id_detalle    INT AUTO_INCREMENT PRIMARY KEY,
    id_pedido     INT NOT NULL,
    id_producto   INT NOT NULL,
    cantidad      INT NOT NULL DEFAULT 1,
    FOREIGN KEY (id_pedido)   REFERENCES pedidos(id_pedido),
    FOREIGN KEY (id_producto) REFERENCES productos(id_producto)
);
```

```sql
CREATE TABLE historial_estados_pedido (
    id_historial    INT AUTO_INCREMENT PRIMARY KEY,
    id_pedido       INT NOT NULL,
    estado_anterior ENUM('RECIBIDA','PREPARANDO','EN_HORNO','LISTA','CANCELADA','NO_ENTREGADO'),
    estado_nuevo    ENUM('RECIBIDA','PREPARANDO','EN_HORNO','LISTA','CANCELADA','NO_ENTREGADO') NOT NULL,
    fecha_cambio    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    origen          ENUM('JUGADOR','SISTEMA') NOT NULL DEFAULT 'JUGADOR',
    FOREIGN KEY (id_pedido) REFERENCES pedidos(id_pedido)
);
```

## Modelo

Clases que representan las entidades de la base de datos, a continuación, se mostrarán algunas relevantes tomando en cuenta que todas tienen la misma sintaxis y estructura ya que lo único que se busca con las clases de modelo es representar las tablas de la base de dato, pero en código, agregándole encapsulamiento, ya que se pueden acceder a ellas únicamente mediante a los constructores, getter y setters utilizados en cada una de las clases modelo.

Clase usada para establecer la conexión de Java con la base de datos.

```
public class ConnectionMySQL {
    private Connection conn;
    private static String url = "jdbc:mysql://localhost:3306/practica1_IPC2";
    private static String usuario = "root";
    private static String password = "1234";

    public Connection conectar() {
        try {
            Class.forName( className: "com.mysql.cj.jdbc.Driver");
            // Establecer la conexión
            conn = DriverManager.getConnection(url,  user: usuario, password);
            System.out.println( x: "Conexión exitosa");
            return conn;
        } catch (SQLException e) {
            System.err.println("Error al conectar: " + e.getMessage());
            return null;
        } catch (ClassNotFoundException e) {
            System.err.println("Error al cargar el driver: " + e.getMessage());
            return null;
        }
    }

    public void desconectar(Connection c) {
        if (c != null) {
            try {
                c.close();
                System.out.println( x: "Conexión cerrada");
            } catch (SQLException e) {
                System.err.println("Error al cerrar conexión: " + e.getMessage());
            }
        }
    }
```

Clase para representar la entidad Usuario.

```java
public class Usuario {
    private int idUsuario;
    private String usuario;
    private String contraseña;
    private String nombre;
    private String apellido;
    private String email;
    private Rol rol;
    private Sucursal sucursal;
    private boolean activo;

    public Usuario(int idUsuario, String usuario,String contraseña, String nombre, String apellido, String email, Rol rol, Sucursal sucursal, boolean activo) {
        this.idUsuario = idUsuario;
        this.usuario = usuario;
        this.contraseña = contraseña;
        this.nombre = nombre;
        this.apellido = apellido;
        this.email = email;
        this.rol = rol;
        this.sucursal = sucursal;
        this.activo = activo;
    }

    public Usuario (){}

    public int getIdUsuario() {
        return idUsuario;
    }

    public String getUsuario() {
        return usuario;
    }

    public String getContraseña(){
        return contraseña;
    }
    public String getNombre() {
        return nombre;
    }

    public String getApellido() {
        return apellido;
    }

    public String getEmail() {
        return email;
    }
```

Clase usada para representar la entidad producto

```java
public class Producto {
    private int idProducto;
    private String nombre;
    private String descripcion;
    private double precio;
    private String categoria;
    private String imagen;
    private boolean activo;

    public Producto (){
    }

    public Producto(int idProducto, String nombre, String descripcion, double precio, String categoria, String imag
        this.idProducto = idProducto;
        this.nombre = nombre;
        this.descripcion = descripcion;
        this.precio = precio;
        this.categoria = categoria;
        this.imagen = imagen;
        this.activo = activo;
    }

    public int getIdProducto() {
        return idProducto;
    }

    public String getNombre() {
        return nombre;
    }

    public String getDescripcion() {
        return descripcion;
    }
```

Clase para representar la entidad partida

```java
public class Partida {
    private int            idPartida;
    private Usuario        usuario;
    private Sucursal       sucursal;
    private LocalDateTime  fechaInicio;
    private LocalDateTime  fechaFin;
    private EstadoPartida  estado;
    private int            puntajeTotal;
    private int            nivelMaximo;
    private int            pedidosCompletados;
    private int            pedidosCancelados;
    private int            pedidosNoEntregados;

    public Partida() {
        this.estado      = EstadoPartida.EN_CURSO;
        this.nivelMaximo = 1;
    }

    public int           getIdPartida()            { return idPartida; }
    public Usuario       getUsuario()              { return usuario; }
    public Sucursal      getSucursal()             { return sucursal; }
    public LocalDateTime getFechaInicio()          { return fechaInicio; }
    public LocalDateTime getFechaFin()             { return fechaFin; }
    public EstadoPartida getEstado()               { return estado; }
    public int           getPuntajeTotal()         { return puntajeTotal; }
    public int           getNivelMaximo()          { return nivelMaximo; }
    public int           getPedidosCompletados()   { return pedidosCompletados; }
    public int           getPedidosCancelados()    { return pedidosCancelados; }
    public int           getPedidosNoEntregados()  { return pedidosNoEntregados; }

    public void setIdPartida(int i)              { this.idPartida            = i; }
    public void setUsuario(Usuario u)            { this.usuario              = u; }
```

## DAO´s

Acceso a datos: operaciones CRUD con PreparedStatement para evitar inyecciones sql, e inclusive inner Join para poder acceder a pedidos por sucursal. Se mostraran algunas clases relevantes para poder observar la estructura de los DAO´S usados en el proyecto que tienen la misma estructura.

Clase ProductoDAO permite la conexión con la base de datos y hace peticiones para poder almacenar, modificar o eliminar productos del sistema, se establece un Inner Join para poder visualizar los atributos de producto y el stock en cada sucursal.

```java
package daos;

import modelo.Producto;
import modelo.ConnectionMySQL;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

/**
 *
 * @author jeffm
 */
public class ProductoDAO {

    ConnectionMySQL connMySQL = new ConnectionMySQL();
    Connection conn = null;

    public ProductoDAO(){
            conn = connMySQL.conectar();
    }

    public Producto obtenerPorID(int id) throws SQLException {
        String sql = "SELECT * FROM productos WHERE id_producto = ?";
        try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt( parameterIndex: 1,  x: id);
            ResultSet rs = ps.executeQuery();
            if (rs.next()) return mapear(rs);
        }
        return null;
    }

    public List<Producto> obtenerTodos() throws SQLException {
        List<Producto> lista = new ArrayList<>();
        String sql = "SELECT * FROM productos ORDER BY nombre";
        try (PreparedStatement ps = conn.prepareStatement(sql);
            ResultSet rs = ps.executeQuery()) {
            while (rs.next()) lista.add( e: mapear(rs));
        }
        return lista;
    }
```

```java
public List<Producto> obtenerDisponiblesPorSucursal(int idSucursal) throws SQLException {
    List<Producto> lista = new ArrayList<>();
    String sql = "SELECT p.* FROM productos p " +
                 "INNER JOIN producto_sucursal ps ON p.id_producto = ps.id_producto " +
                 "WHERE ps.id_sucursal = ? AND ps.disponible = TRUE AND p.activo = TRUE " +
                 "AND ps.stock > 0 " +
                 "ORDER BY p.nombre";
    try (PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setInt( parameterIndex: 1,  x: idSucursal);
        ResultSet rs = ps.executeQuery();
        while (rs.next()) lista.add( e: mapear(rs));
    }
    return lista;
}

public boolean ingresar(Producto p) throws SQLException {
    String sql = "INSERT INTO productos (nombre, descripcion, precio, categoria, imagen, activo) VALUES (?,?,?,?,?,?,?)";
    try (PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString( parameterIndex: 1,  x: p.getNombre());
        ps.setString( parameterIndex: 2,  x: p.getDescripcion());
        ps.setDouble( parameterIndex: 3,  x: p.getPrecio());
        ps.setString( parameterIndex: 4,  x: p.getCategoria());
        ps.setString( parameterIndex: 5,  x: p.getImagen());
        ps.setBoolean( parameterIndex: 6,  x: p.isActivo());
        int rows = ps.executeUpdate();
        if (rows > 0) {
            ResultSet keys = ps.getGeneratedKeys();
            if (keys.next()) {
                p.setIdProducto( idProducto: keys.getInt( columnIndex: 1)); // ← asignar ID
            }
        }
        return rows > 0;
    }
}
```

Clase de UsuarioDAO, nos permite comunicarnos para poder registrar usuarios y también para poder desactivarlos (banearlos) o actualizar información y/o a qué sucursal pertenecen.

```java
package daos;

import modelo.Rol;
import modelo.Sucursal;
import modelo.Usuario;
import modelo.ConnectionMySQL;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;
/**
 *
 * @author jeffm
 */
public class UsuarioDAO {
    private final RolDAO      rolDAO      = new RolDAO();
    private final SucursalDAO sucursalDAO = new SucursalDAO();

    ConnectionMySQL connMySQL = new ConnectionMySQL();
    Connection conn = null;

    public UsuarioDAO(){
            conn = connMySQL.conectar();
    }

    public Usuario encontrarPorId(int id) throws SQLException {
        String sql = "SELECT * FROM usuarios WHERE id_usuario = ?";
        try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt( parameterIndex: 1,  x: id);
            ResultSet rs = ps.executeQuery();
            if (rs.next()) return mapear(rs);
        }
        return null;
    }
```

```java
public boolean existeUsuario(String username) throws SQLException {
    String sql = "SELECT COUNT(*) FROM usuarios WHERE usuario = ?";
    try (PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString( parameterIndex: 1,  x:username);
        ResultSet rs = ps.executeQuery();
        if (rs.next()) return rs.getInt( columnIndex: 1) > 0;
    }
    return false;
}


public List<Usuario> encontrarTodos() throws SQLException {
    List<Usuario> lista = new ArrayList<>();
    String sql = "SELECT * FROM usuarios WHERE activo = TRUE ORDER BY nombre";
    try (PreparedStatement ps = conn.prepareStatement(sql);
        ResultSet rs = ps.executeQuery()) {
        while (rs.next()) lista.add( e:mapear(rs));
    }
    return lista;
}

public boolean ingresar(Usuario u) throws SQLException {
    String sql = "INSERT INTO usuarios (usuario, contraseña, nombre, apellido, email, id_rol, id_sucursal, activo) VALUES (?,?,?,?,?,?,
    try (PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString( parameterIndex: 1,  x:u.getUsuario());
        ps.setString( parameterIndex: 2,  x:u.getContraseña());
        ps.setString( parameterIndex: 3,  x:u.getNombre());
        ps.setString( parameterIndex: 4,  x:u.getApellido());
        ps.setString( parameterIndex: 5,  x:u.getEmail());
        ps.setInt( parameterIndex: 6,  x:u.getRol().getIdRol());
        if (u.getSucursal() != null) ps.setInt( parameterIndex: 7,  x:u.getSucursal().getIdSucursal());
        else                         ps.setNull( parameterIndex: 7,  sqlType: Types.INTEGER);
        ps.setBoolean( parameterIndex: 8,  x:true);
        return ps.executeUpdate() > 0;
    }
}
```

Clase Sucursal DAO para poder crear, actualizar, o eliminar sucursales en el sistema.

```java
package daos;

import modelo.Sucursal;
import modelo.ConnectionMySQL;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

/**
 *
 * @author jeffm
 */
public class SucursalDAO {
    ConnectionMySQL connMySQL = new ConnectionMySQL();
    Connection conn = null;

    public SucursalDAO() {
        conn = connMySQL.conectar();

    }

    public Sucursal obtenerSucursal(int id) throws SQLException {
        String sql = "SELECT * FROM sucursales WHERE id_sucursal = ?";
        try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt( parameterIndex: 1,  x:id);
            ResultSet rs = ps.executeQuery();
            if (rs.next()) return mapear(rs);
        }
        return null;
    }

    public List<Sucursal> obtenerTodas() throws SQLException {
        List<Sucursal> lista = new ArrayList<>();
        String sql = "SELECT * FROM sucursales WHERE activa = TRUE ORDER BY nombre";
        try (PreparedStatement ps = conn.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
```

```java
public boolean ingresar(Sucursal s) throws SQLException {
    String sql = "INSERT INTO sucursales (nombre, direccion, activa) VALUES (?,?,?)";
    try (PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString( parameterIndex: 1,  x: s.getNombre());
        ps.setString( parameterIndex: 2,  x: s.getDirección());
        ps.setBoolean( parameterIndex: 3,  x: s.isActiva());
        return ps.executeUpdate() > 0;
    }
}

public boolean actualizar(Sucursal s) throws SQLException {
    String sql = "UPDATE sucursales SET nombre=?, direccion=?, activa=? WHERE id_sucursal=?";
    try (PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString( parameterIndex: 1,  x: s.getNombre());
        ps.setString( parameterIndex: 2,  x: s.getDirección());
        ps.setBoolean( parameterIndex: 3,  x: s.isActiva());
        ps.setInt( parameterIndex: 4,  x: s.getIdSucursal());
        return ps.executeUpdate() > 0;
    }
}

public boolean eliminar(int id) throws SQLException {
    String sql = "UPDATE sucursales SET activa = FALSE WHERE id_sucursal = ?";
    try (PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setInt( parameterIndex: 1,  x: id);
        return ps.executeUpdate() > 0;
    }
}

private Sucursal mapear(ResultSet rs) throws SQLException {
    return new Sucursal(
        idSucursal: rs.getInt( columnLabel: "id_sucursal"),
        nombre: rs.getString( columnLabel: "nombre"),
        dirección: rs.getString( columnLabel: "direccion"),
        activa: rs.getBoolean( columnLabel: "activa")
    );
```

## Controladores

Lógica de negocio, validaciones y reglas del juego, el motor del juego para que se pueda desarrollar sin problemas, en este sistema particular se hacen uso de 4 importantes controladores los cuales se van a plantear algunos de los métodos más importantes de cada controlador.

**Login Controlador** es importante para la lógica del inicio de sesión y verificar el rol que tiene cada usuario en determinada sesión existente dentro del sistema.

```java
import daos.UsuarioDAO;
import modelo.Usuario;
import modelo.Constantes;
import java.sql.SQLException;
/**
 *
 * @author jeffm
 */
public class LoginControlador {
     private static Usuario sesionActual = null;

    private final UsuarioDAO usuarioDAO = new UsuarioDAO();

    public Usuario login(String usuario, String contraseña) throws SQLException {
        Usuario user = usuarioDAO.encontrarPorUsuario(usuario, contraseña);
        if (user == null) return null;
        sesionActual = user;
        return user;
    }

    public void logout() {
        sesionActual = null;
    }

    public static Usuario getSesionActual() {
        return sesionActual;
    }

    public static boolean esSuperAdmin() {
        return sesionActual != null &&
                Constantes.ROL_SUPER_ADMIN.equals( anObject: sesionActual.getRol().getNombre());
    }

    public static boolean esAdminTienda() {
        return sesionActual != null &&
                Constantes.ROL_ADMIN_TIENDA.equals( anObject: sesionActual.getRol().getNombre());
    }

    public static boolean esJugador() {
        return sesionActual != null &&
                Constantes.ROL_JUGADOR.equals( anObject: sesionActual.getRol().getNombre());
    }
}
```

El **controlador del Juego** es importante para verificar la lógica del juego como tal y permite verificar validaciones para poder avanzar entre estados y generar reglas para que el juego se pueda llevar de la manera más optima posible

```java
import daos.*;
import modelo.*;
import modelo.enums.*;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;
import java.util.Random;

/**
 *
 * @author jeffm
 */
public class JuegoControlador {
    private final PartidaDAO             partidaDAO    = new PartidaDAO();
    private final PedidoDAO              pedidoDAO     = new PedidoDAO();
    private final DetallePedidoDAO       detalleDAO    = new DetallePedidoDAO();
    private final HistorialEstadoPedidoDAO historialDAO = new HistorialEstadoPedidoDAO();
    private final NivelPartidaDAO        nivelDAO      = new NivelPartidaDAO();
    private final ParametroJuegoDAO      paramDAO      = new ParametroJuegoDAO();
    private final ProductoDAO            productoDAO   = new ProductoDAO();
    private final ProductoSucursalDAO    psDao         = new ProductoSucursalDAO();

    private Partida        partida;
    private List<Pedido>   pedidosActivos;
    private int            nivelActual;
    private int            turnoRestante;
    private int            contadorPedidos;
    private int            segundosSinGenerar;
    private final Random   random = new Random();

    // Parametros cargados de BD
    private int maxPedidosActivos;
    private int duracionTurno;
    private int tiempoNivel1, tiempoNivel2, tiempoNivel3;
    private int puntosOk, puntosBonus, puntosCancelado, puntosNoEntregado;
    private int pedidosNivel2, pedidosNivel3;
```

El método de iniciar partida es clave porque es donde todo empieza, por ello nos permite cargar los parametros del juego, así como los productos con stock disponibles en cada sucursal, y practicamente se ponen todos los valores por defecto.

```java
public void iniciarPartida(Usuario jugador) throws SQLException {
    cargarParametros();
    List<Producto> disponibles = productoDAO.obtenerDisponiblesPorSucursal(
            idSucursal: jugador.getSucursal().getIdSucursal());
    if (disponibles.isEmpty()) {
        throw new IllegalStateException(
                "No hay productos con stock disponible en esta sucursal.\n"
                + "El administrador debe reponer el stock antes de jugar.");
    }
    pedidosActivos    = new ArrayList<>();
    nivelActual       = 1;
    turnoRestante     = duracionTurno;
    contadorPedidos   = 0;
    segundosSinGenerar = intervaloGeneracion;

    partida = new Partida();
    partida.setUsuario( u: jugador);
    partida.setSucursal( s: jugador.getSucursal());
    partidaDAO.ingresar( p: partida);

    maxPedidosActivosActual = maxPedidosActivos;
    intervaloActual = intervaloGeneracion;

    // Registrar nivel 1
    registrarNivel( nivel: 1);
}
```

Este método permite que el usuario avance entre los diferentes estados de los pedidos, por medio de clicks en un botón, y de esta manera poder avanzar y entregar los pedidos que se indiquen. Tiene diferentes validaciones dependiendo del estado en el que se encuentre.

```java
public String avanzarEstado(Pedido pedido) throws SQLException {
    if (pedido.getEstado().esFinalizado())
        return "Este pedido ya esta finalizado.";

    if (pedido.estaBloqueado())
        return "Espera " + pedido.getTiempoBloqueo() + "s para avanzar este pedido.";

    EstadoPedido siguiente = pedido.getEstado().siguiente();
    if (siguiente == null) return "No hay estado siguiente.";

    EstadoPedido anterior = pedido.getEstado();
    cambiarEstado( p: pedido,  nuevo: siguiente,  origen: Origen.JUGADOR);

    if (siguiente == EstadoPedido.LISTA) {

        for (modelo.DetallePedido det : pedido.getDetalles()){
            psDao.decrementarStock( idProducto: det.getProducto().getIdProducto(),  idSucursal: partida.getSucursal().getIdSucursal(),
        }

        int tiempoUsado = pedido.getTiempoLimiteDeg() - pedido.getTiempoRestante();
        pedido.setTiempoUsado(tiempoUsado);
        int puntos = calcularPuntos( p: pedido);
        pedido.setPuntosObtenidos( puntosObtenidos: puntos);
        partida.sumarPuntos(puntos);
        partida.incrementarCompletados();
        pedidoDAO.actualizar( p: pedido);
        verificarSubidaNivel();
        return "Pedido #" + pedido.getNumeroPedido() + " entregado! +" + puntos + " puntos";
    }

    int numProductos = pedido.getDetalles().size();
    int baseBloqueo = 4 + random.nextInt( bound: 9); // 4 a 12 segundos
    int extraPorProductos = Math.max( a: 0, numProductos - 1) * 2;
    pedido.setTiempoBloqueo(baseBloqueo + extraPorProductos);

    return "Pedido #" + pedido.getNumeroPedido() + " avanzado a " + siguiente.name();
}

public String cancelarPedido(Pedido pedido) throws SQLException {
```

Este último metodo es importante ya que es el que se encarga de la generación de pedidos durante el transcurso del juego, mantiene un List con los productos con stock existentes para poder generar los pedidos, se usa un random para poder realizar pedidos desde 1 producto hasta el largo del listado de productos.

```
private void generarPedidoAleatorio() throws SQLException {
    List<Producto> disponibles = productoDAO.obtenerDisponiblesPorSucursal(
        idSucursal: partida.getSucursal().getIdSucursal());

    if (disponibles.isEmpty()) return;

    contadorPedidos++;
    Pedido p = new Pedido();
    p.setIdPartida( idPartida: partida.getIdPartida());
    p.setNumeroPedido( numeroPedido: contadorPedidos);
    p.setNivelAlCrear( nivelAlCrear: nivelActual);
    int tiempoPedido = getTiempoSegunNivel();
    p.setTiempoLimiteDeg( tiempoLimiteDeg: tiempoPedido);
    p.setTiempoRestante( tiempoRestante: tiempoPedido);
    pedidoDAO.ingresar(p);

    // Agregar entre 1 y la  cantidad de productos existentes con stock y activos aleatorios
    int cantProductos = 1 + random.nextInt( bound: disponibles.size());
    List<Producto> seleccionados = new ArrayList<>();
    for (int i = 0; i < cantProductos; i++) {
        Producto prod = disponibles.get( index: random.nextInt( bound: disponibles.size()));
        if (!seleccionados.contains( o: prod)) {
            seleccionados.add( e: prod);
            DetallePedido det = new DetallePedido( idPedido: p.getIdPedido(),  producto: prod,  cantidad: 1);
            detalleDAO.ingresar( d: det);
            p.getDetalles().add( e: det);
        }
    }

    registrarHistorial(p,  anterior: null,  nuevo: EstadoPedido.RECIBIDA,  origen: Origen.SISTEMA);
    pedidosActivos.add( e: p);
}
```

El **controlador del administrador de tiendas** permite la creacion de productos para la sucursal de la cual son encargados, así como de la supervision del stock de los productos y la exportación de los reportes en formato CSV.

```java
public class AdminControlador {
    private final ProductoDAO        productoDAO        = new ProductoDAO();
    private final ProductoSucursalDAO psDAO             = new ProductoSucursalDAO();
    private final PartidaDAO         partidaDAO         = new PartidaDAO();


    //Manejar productos en cada sucursal
    public List<Producto> obtenerTodosProductos() throws SQLException {
        return productoDAO.obtenerTodos();
    }

    public List<Producto> obtenerProductosSucursal(int idSucursal) throws SQLException {
        return productoDAO.obtenerDisponiblesPorSucursal(idSucursal);
    }

    public boolean crearProducto(String nombre, String descripcion, double precio,
                                 String categoria, int idSucursal, int stock) throws SQLException {
        if (nombre == null || nombre.trim().isEmpty()) return false;
        Producto p = new Producto();
        p.setNombre( nombre: nombre.trim());
        p.setDescripcion(descripcion);
        p.setPrecio(precio);
        p.setCategoria(categoria);
        p.setActivo( activo: true);
        boolean ok = productoDAO.ingresar(p);
        // Se asocia a la sucursal actual
        if (ok) {
            ProductoSucursal ps = new ProductoSucursal();
            ps.setProducto( producto: p);
            Sucursal s = new Sucursal();
            s.setIdSucursal(idSucursal);
            ps.setSucursal( sucursal: s);
            ps.setDisponible( disponible: true);
            psDAO.ingresar( ps2: ps);
        }
        return ok;
    }
```

```java
    public boolean actualizarProducto(Producto p) throws SQLException {
        return productoDAO.actualizar(p);
    }
    public boolean actualizarStockSucursal(ProductoSucursal ps) throws SQLException {
    return psDAO.actualizar( ps2: ps);
    }

    public boolean desactivarProducto(int id) throws SQLException {
        return productoDAO.eliminar(id);
    }

    public boolean toggleDisponibilidad(int idProducto, int idSucursal, boolean disponible) throws SQLException {
        return psDAO.toggleDisponible(idProducto, idSucursal, disponible);
    }

    public boolean decrementarStock(int idProducto, int idSucursal, int cantidad) throws SQLException {
        return psDAO.decrementarStock(idProducto, idSucursal, cantidad);
    }

    // Muestra las estadiscticas de cada sucursal
    public List<Partida> obtenerPartidasSucursal(int idSucursal) throws SQLException {
        return partidaDAO.obtenerFinalizadasPorSucursal(idSucursal);
    }

    public void exportarCSV(List<Partida> partidas, String rutaArchivo) throws IOException {
        try (PrintWriter pw = new PrintWriter(new FileWriter( fileName: rutaArchivo))) {
            pw.println( x: "ID,Jugador,Sucursal,Puntaje,Nivel,Completados,Cancelados,NoEntregados");
            for (Partida p : partidas) {
                pw.printf( format: "%d,%s,%s,%d,%d,%d,%d,%d%n",
                     args: p.getIdPartida(),
                    p.getUsuario() != null ? p.getUsuario().getNombreCompleto() : "",
                    p.getSucursal() != null ? p.getSucursal().getNombre() : "",
                     args: p.getPuntajeTotal(),
                     args: p.getNivelMaximo(),
                     args: p.getPedidosCompletados(),
                     args: p.getPedidosCancelados(),
                     args: p.getPedidosNoEntregados()
                );
            }
        }
    }
```

El **control del Super Administrador** permite la creación de sucursales y también la creación y registros de usuarios, por lo que es importante tener clara está logica de servicios y que se encuentre funcionando correctamente, porque directamente acá van todos los usuarios que pueden ser Jugadores o Administradores de Tiendas.

```java
public class SuperAdminControlador {

    private final UsuarioDAO        usuarioDAO        = new UsuarioDAO();
    private final SucursalDAO       sucursalDAO       = new SucursalDAO();
    private final RolDAO            rolDAO            = new RolDAO();
    private final ParametroJuegoDAO parametroDAO      = new ParametroJuegoDAO();

    // Manejo de las sucursales en global
    public List<Sucursal> obtenerSucursales() throws SQLException {
        return sucursalDAO.obtenerTodas();
    }

    public boolean crearSucursal(String nombre, String direccion) throws SQLException {
        if (nombre == null || nombre.trim().isEmpty()) return false;
        Sucursal s = new Sucursal();
        s.setNombre( nombre: nombre.trim());
        s.setDirección( dirección: direccion);
        s.setActiva( activa: true);
        return sucursalDAO.ingresar(s);
    }

    public boolean actualizarSucursal(Sucursal s) throws SQLException {
        return sucursalDAO.actualizar(s);
    }

    public boolean desactivarSucursal(int id) throws SQLException {
        return sucursalDAO.eliminar(id);
    }

    // Manejo de usuarios en Global
    public List<Usuario> obtenerUsuarios() throws SQLException {
        return usuarioDAO.encontrarTodos();
    }
```

```java
public boolean registrarUsuario(String usuario, String contraseña, String nombre,
                                String apellido, String email, int idRol,
                                Integer idSucursal) throws SQLException {
    if (usuarioDAO.existeUsuario( username: usuario))
        throw new IllegalArgumentException("El usuario '" + usuario + "' ya existe.");

    Usuario u = new Usuario();
    u.setUsuario( usuario: usuario.trim());
    u.setContraseña(contraseña);
    u.setNombre( nombre: nombre.trim());
    u.setApellido(apellido);
    u.setEmail(email);
    u.setRol( rol: rolDAO.encontrarPorId( id: idRol));
    if (idSucursal != null) u.setSucursal( sucursal: new SucursalDAO().obtenerSucursal( id: idSucursal))
    u.setActivo( activo: true);
    return usuarioDAO.ingresar(u);
}

public boolean desactivarUsuario(int id) throws SQLException {
    return usuarioDAO.eliminar(id);
}

public boolean cambiarSucursalUsuario(int idUsuario, Integer idSucursal) throws SQLException {
    return usuarioDAO.cambiarSucursal(idUsuario, idSucursal);
}

//Obtener y cambiar los parametros del juego en global
public List<ParametroJuego> obtenerParametros() throws SQLException {
    return parametroDAO.obtenerTodos();
}

public boolean guardarParametro (ParametroJuego p) throws SQLException {
    return parametroDAO.ingresar(p);
}

public boolean actualizarParametro(ParametroJuego p) throws SQLException {
    return parametroDAO.actualizar(p);
}

// ver los roles disponibles
public List<Rol> obtenerRoles() throws SQLException {
    return rolDAO.encontrarTodos();
}
```

## Interfaz gráfica - Vista

La interfaz gráfica está implementada utilizando Swing, una biblioteca estándar de Java para interfaces gráficas. La clase principal es la **del Login**, que extiende de JFrame. En esta vista encontramos el formulario correspondiente para poder ingresar las credenciales de acceso al sistema, o bien para poder crear una cuenta.

```java
public class LoginVista extends JFrame{
     private JTextField     txtUsername;
    private JPasswordField txtPassword;
    private JButton        btnLogin;
    private JButton        btnRegistro;
    private JLabel         lblMensaje;

    public LoginVista() {
        initComponents();
        setVisible(b:true);
    }

    private void initComponents() {
        setTitle(title:"Pizza Express Tycoon - Login");
        setSize(width:550, height:450);
        setDefaultCloseOperation(operation:EXIT_ON_CLOSE);
        setLocationRelativeTo(c:null);
        setResizable(resizable:false);

        JPanel panelPrincipal = new JPanel(new BorderLayout());
        panelPrincipal.setBackground(new Color(r:44, g:62, b:80));

        // — Encabezado ————————————————————————
        JLabel lblTitulo = new JLabel(text:"Pizza Express Tycoon", horizontalAlignment:SwingConstants.CENTER);
        lblTitulo.setFont(new Font(name:"Arial", style:Font.BOLD, size:22));
        lblTitulo.setForeground(fg:Color.WHITE);
        lblTitulo.setBorder(border:BorderFactory.createEmptyBorder(top:20, left:10, bottom:10, right:10));
        panelPrincipal.add(comp:lblTitulo, constraints:BorderLayout.NORTH);
```

```java
// — Formulario ————————————————————————————
JPanel panelForm = new JPanel(new GridBagLayout());
panelForm.setBackground(new Color(r:44, g:62, b:80));
panelForm.setBorder(border:BorderFactory.createEmptyBorder(top:10, left:40, bottom:10, right:40));

GridBagConstraints gbc = new GridBagConstraints();
gbc.insets = new Insets(top:8, left:5, bottom:8, right:5);
gbc.fill   = GridBagConstraints.HORIZONTAL;

// Campo usuario
gbc.gridx = 0; gbc.gridy = 0;
JLabel lblUser = new JLabel(text:"Usuario:");
lblUser.setForeground(fg:Color.WHITE);
panelForm.add(comp:lblUser, constraints:gbc);

gbc.gridx = 1;
txtUsername = new JTextField(columns:16);
panelForm.add(comp:txtUsername, constraints:gbc);

// Campo contraseña
gbc.gridx = 0; gbc.gridy = 1;
JLabel lblPass = new JLabel(text:"Contraseña:");
lblPass.setForeground(fg:Color.WHITE);
panelForm.add(comp:lblPass, constraints:gbc);

gbc.gridx = 1;
txtPassword = new JPasswordField(columns:16);
panelForm.add(comp:txtPassword, constraints:gbc);

// Boton iniciar sesion
gbc.gridx = 0; gbc.gridy = 2; gbc.gridwidth = 2;
btnLogin = new JButton(text:"Iniciar Sesion");
btnLogin.setBackground(new Color(r:231, g:76, b:60));
btnLogin.setForeground(fg:Color.BLACK);
btnLogin.setFont(new Font(name:"Arial", style:Font.BOLD, size:14));
btnLogin.setFocusPainted(b:false);
panelForm.add(comp:btnLogin, constraints:gbc);
```

Este método es el que permite el acceso al sistema. Validando si las credenciales coinciden con las almacenadas en la base de datos.

```java
private void realizarLogin() {
    String username = txtUsername.getText().trim();
    String password = new String( value: txtPassword.getPassword());

    if (username.isEmpty() || password.isEmpty()) {
        lblMensaje.setText( text: "Ingresa usuario y contraseña.");
        return;
    }

    try {
        LoginControlador auth = new LoginControlador();
        Usuario usuario = auth.login( usuario: username,  contraseña: password);
        if (usuario == null) {
            lblMensaje.setText( text: "Credenciales incorrectas.");
        } else {
            dispose();
            new MenuPrincipalVista();
        }
    } catch (SQLException ex) {
        lblMensaje.setText( text: "Error de conexion a la base de datos.");
        ex.printStackTrace();
    }
}

private void abrirRegistro() {
    dispose();
    new RegistroVista();
}
```

La clase de **JuegoVista** es la principal porque es donde se desarrolla el juego, todo gira en torno a ella ya que sin ella, no habrian rankings, historial, ni se mostrarian productos como tal. Y se mostrarán uno de los métodos mas importantes que almacena está clase.

```java
public class JuegoVista extends JFrame{
    private final JuegoControlador controller = new JuegoControlador();
    private final JFrame           menuParent;

    private JLabel     lblTurno;
    private JLabel     lblPuntaje;
    private JLabel     lblNivel;
    private JLabel     lblMensaje;
    private JPanel     panelPedidos;
    private JScrollPane scrollPedidos;
    private javax.swing.Timer timerJuego;
    private javax.swing.Timer timerMensaje;

    public JuegoVista(JFrame menuParent) {
        this.menuParent = menuParent;
        initComponents();
        iniciarJuego();
    }

    private void initComponents() {
        setTitle( title: "Pizza Express ");
        setSize( width: 900,  height: 620);
        setDefaultCloseOperation( operation: DO_NOTHING_ON_CLOSE);
        setLocationRelativeTo( c: null);
        setResizable( resizable: false);

        JPanel panelPrincipal = new JPanel(new BorderLayout( hgap: 5,  vgap: 5));
        panelPrincipal.setBackground(new Color( r: 44,  g: 62,  b: 80));
        panelPrincipal.setBorder( border: BorderFactory.createEmptyBorder( top: 8,  left: 8,  bottom: 8,  right: 8));

        // Panel superior:
        JPanel panelHUD = new JPanel(new FlowLayout( align: FlowLayout.CENTER,  hgap: 40,  vgap: 6));
        panelHUD.setBackground(new Color( r: 52,  g: 73,  b: 94));
        panelHUD.setBorder( border: BorderFactory.createLineBorder(new Color( r: 231,  g: 76,  b: 60),  thickness: 2));
```

```java
//Panel central: pedidos activos
panelPedidos = new JPanel();
panelPedidos.setLayout(new BoxLayout( target: panelPedidos,  axis: BoxLayout.Y_AXIS));
panelPedidos.setBackground(new Color( r: 44,  g: 62,  b: 80));

lblMensaje = new JLabel( text: " ",  horizontalAlignment: SwingConstants.CENTER);
lblMensaje.setFont(new Font( name: "Arial",  style: Font.BOLD,  size: 14));
lblMensaje.setForeground(new Color( r: 39,  g: 174,  b: 96));
lblMensaje.setOpaque( isOpaque: true);
lblMensaje.setBackground(new Color( r: 52,  g: 73,  b: 94));
lblMensaje.setBorder( border: BorderFactory.createEmptyBorder( top: 4,  left: 10,  bottom: 4,  right: 10));

scrollPedidos = new JScrollPane( view: panelPedidos);
scrollPedidos.setBorder( border: BorderFactory.createTitledBorder(
    border: BorderFactory.createLineBorder( color: Color.GRAY),  title: "Pedidos Activos",
    titleJustification: 0,  titlePosition: 0,  new Font( name: "Arial",  style: Font.BOLD,  size: 14),  titleColor: Color.WHITE));
scrollPedidos.getViewport().setBackground(new Color( r: 44,  g: 62,  b: 80));
 JPanel panelCentro = new JPanel(new BorderLayout( hgap: 0,  vgap: 4));
panelCentro.setBackground(new Color( r: 44,  g: 62,  b: 80));
panelCentro.add( comp: lblMensaje,  constraints: BorderLayout.NORTH);
panelCentro.add( comp: scrollPedidos,  constraints: BorderLayout.CENTER);
panelPrincipal.add( comp: panelCentro,  constraints: BorderLayout.CENTER);

// Boton finzalizar partida
JButton btnRendirse = new JButton( text: "Finalizar Partida");
btnRendirse.setBackground(new Color( r: 192,  g: 57,  b: 43));
btnRendirse.setForeground( fg: Color.BLACK);
btnRendirse.setFont(new Font( name: "Arial",  style: Font.BOLD,  size: 13));
btnRendirse.setFocusPainted( b: false);
btnRendirse.addActionListener(e -> confirmarFinalizacion());
JPanel panelSur = new JPanel(new FlowLayout( align: FlowLayout.RIGHT));
panelSur.setBackground(new Color( r: 44,  g: 62,  b: 80));
panelSur.add( comp: btnRendirse);
panelPrincipal.add( comp: panelSur,  constraints: BorderLayout.SOUTH);

add( comp: panelPrincipal);
setVisible( b: true);
```

El método de iniciar juego permite verificar el stock de los productos antes de iniciar una partida, también inicializa el timer, y la generación de pedidos

```java
private void iniciarJuego() {
    try {
        controller.iniciarPartida( jugador: LoginControlador.getSesionActual());
    } catch (IllegalStateException e) {
        JOptionPane.showMessageDialog( parentComponent: this,
                message: e.getMessage(),
                title: "Sin stock disponible",
                messageType: JOptionPane.WARNING_MESSAGE);
        dispose();
        menuParent.setVisible( b: true);
        return;
    }catch (SQLException e) {
        JOptionPane.showMessageDialog( parentComponent: this, "Error al iniciar la partida: " + e.getMessage());
        dispose();
        menuParent.setVisible( b: true);
        return;
    }

    timerJuego = new javax.swing.Timer( delay: 1000, e -> {
        try {
            controller.tick();
            actualizarHUD();
            actualizarPedidos();
            if (controller.getPartida() != null &&
                controller.getTurnoRestante() <= 0) {
                timerJuego.stop();
                mostrarFinPartida();
            }
        } catch (SQLException ex) {
            ex.printStackTrace();
        }
    });
    timerJuego.start();
}
```

El método de pedidoPanel, es crucial ya que acá se muestra al usuario los pedidos, así como también toda su informacion y los botones para poder avanzar o cancelar los pedidos.

```java
// Panel de cada pedido
private class PedidoPanel extends JPanel {

    private final Pedido pedido;

    PedidoPanel(Pedido pedido) {
        this.pedido = pedido;
        setLayout(new BorderLayout( hgap: 8,  vgap: 4));
        setBorder( border: BorderFactory.createCompoundBorder(
            outsideBorder: BorderFactory.createLineBorder( color: colorEstado( estado: pedido.getEstado()),  thickness: 2),
            insideBorder: BorderFactory.createEmptyBorder( top: 8,  left: 10,  bottom: 8,  right: 10)
        ));
        setBackground(new Color( r: 52,  g: 73,  b: 94));
        setMaximumSize(new Dimension( width: Integer.MAX_VALUE,  height: 150));

        // Info izquierda
        JPanel infoPanel = new JPanel(new GridLayout( rows: 3,  cols: 1,  hgap: 0,  vgap: 2));
        infoPanel.setOpaque( isOpaque: false);

        JLabel lblNum = new JLabel("Pedido #" + pedido.getNumeroPedido() +
            "  |  Estado: " + pedido.getEstado().name());
        lblNum.setFont(new Font( name: "Arial",  style: Font.BOLD,  size: 13));
        lblNum.setForeground( fg: Color.black);

        JLabel lblProductos = new JLabel( text: pedido.getResumenProducto());
        lblProductos.setFont(new Font( name: "Arial",  style: Font.PLAIN,  size: 12));
        lblProductos.setForeground(new Color( r: 189,  g: 195,  b: 199));

        Color timerColor = pedido.getTiempoRestante()<= 10
            ? new Color( r: 231,  g: 76,  b: 60) : new Color( r: 241,  g: 196,  b: 15);
        JLabel lblTimer = new JLabel("Tiempo restante: " + pedido.getTiempoRestante() + "s");
        lblTimer.setFont(new Font( name: "Arial",  style: Font.BOLD,  size: 12));
        lblTimer.setForeground( fg: timerColor);
```

```java
    // Botones derecha
    JPanel panelBtns = new JPanel(new GridLayout( rows: 1,  cols: 2,  hgap: 5,  vgap: 0));
    panelBtns.setOpaque( isOpaque: false);
    panelBtns.setPreferredSize(new Dimension( width: 250,  height: 90));


    boolean bloqueado = pedido.estaBloqueado();
    String textoAvanzar = bloqueado ? "Avanzar (" + pedido.getTiempoBloqueo() + "s)" : "Avanzar";

    JButton btnAvanzar = new JButton( text: textoAvanzar);
    btnAvanzar.setBackground(new Color( r: 39,  g: 174,  b: 96));
    btnAvanzar.setForeground( fg: Color.BLACK);
    btnAvanzar.setFocusPainted( b: false);
    btnAvanzar.setEnabled(!pedido.getEstado().esFinalizado());
    btnAvanzar.addActionListener(e -> {
        try {
            String msg = controller.avanzarEstado(pedido);
            mostrarMensaje( texto: msg,  esError: false);
            actualizarPedidos();
        } catch (SQLException ex) { ex.printStackTrace(); }
    });

    JButton btnCancelar = new JButton( text: "Cancelar");
    btnCancelar.setBackground(new Color( r: 192,  g: 57,  b: 43));
    btnCancelar.setForeground( fg: Color.BLACK);
    btnCancelar.setFocusPainted( b: false);
    btnCancelar.setEnabled( b: pedido.getEstado().sePuedeCancelar());
    btnCancelar.addActionListener(e -> {
        try {
            int op = JOptionPane.showConfirmDialog( parentComponent: JuegoVista.this,
                "Cancelar pedido #" + pedido.getNumeroPedido() + "?",
                 title: "Cancelar pedido",  optionType: JOptionPane.YES_NO_OPTION);
            if (op == JOptionPane.YES_OPTION) {
                timerJuego.stop();
                controller.cancelarPedido(pedido);
                actualizarPedidos();
```

**Componentes principales de las distintas vistas**

- **Editor de texto**: Implementado con JTextField, permite escribir y rellenar campos de información
- **Contraseña**: Un JPasswordField que permite ingresar una contraseña de manera segura
- **Tablas de resultados**: Implementadas con JTable, muestran información como por ejemplo de Usuarios registrados, Sucursales creadas, los rankings, y las distintas estadisticas.
- **Barra de herramientas**: Contiene botones para las operaciones comunes.